RAJALAKSHMI ENGINEERING COLLEGE

Department of Computer Science and Engineering (Cyber Security)

CS23533– Foundations of Artificial Intelligence

2026-2027

REVIEW 2

AI-Based Cyber Defence Allocation Using Game Theory

Supervisor
Dr. Bhuvana S

Presented by
Puvanesu R
241901085

Problem Statement and Motivation

- The game addresses network defence, where a limited security budget must be spread across many vulnerable digital assets.
- Attackers pick off whatever stays exposed, so every protect, scan or pass decision reshapes the whole risk landscape.
- The project shows how adversarial search, Minimax and Alpha-Beta pruning, models a real attacker-defender struggle.
- A live worst-case damage forecast, search statistics and a reasoning trail make every AI decision easy to inspect.

Objectives

- Model the attack-defence struggle as a two-player, zero-sum, turn-taking game on a board of assets.
- Give the Defender budgeted protect, scan and pass actions against a damage-maximising Attacker.
- Implement Minimax with Alpha-Beta pruning, high-risk-first ordering and a transposition table.
- Forecast worst-case damage live during play with node, cutoff and cache-hit statistics.
- Compare both algorithms on the same scenarios and explain every move on the game console.

Project Progress Since Review 1

- Review 1 baseline (September 2026): literature survey, problem identification, methodology, core implementation (game rules, Minimax, Alpha-Beta, console) and passing play tests.
- Since Review 1: re-ran seeded matches and the algorithm comparison on the current code to confirm the results.
- Documented the design with a Level-0 DFD, development and runtime workflow diagrams and module-level notes.
- Added measured comparisons: Minimax 85,944 vs Alpha-Beta 3,644 nodes on an 8-asset start state (95.8% fewer).
- Remaining: bigger-board benchmarks, difficulty comparisons, screenshots and final documentation.

Literature Survey

No.	Author(s) & Year	Paper Title	Methodology / Algorithm	Limitations
1	Russell & Norvig (2021)	Artificial Intelligence: A Modern Approach (4th ed.)	Minimax with Alpha-Beta pruning, evaluation functions, game theory	Textbook two-player forms. The game adapts them to budgeted security defence.
2	Shannon (1950)	Programming a Computer for Playing Chess	Minimax search with an evaluation function over game trees	Fixed-depth chess evaluation. Budgets and asset risk are not modeled.
3	Lye & Wing (2005)	Game Strategies in Network Security	Two-player game model of attacker versus network administrator	Continuous stochastic models; hard to solve exactly and not playable.

Literature Survey

No.	Author(s) & Year	Paper Title	Methodology / Algorithm	Limitations
4	Knuth & Moore (1975)	An Analysis of Alpha-Beta Pruning	Alpha-Beta pruning effectiveness with move ordering	Savings depend on ordering. The game tries high-risk assets first.
5	Slate & Atkin (1977)	CHESS 4.5 – The Northwestern University Chess Program	Transposition table caching subtree values for reuse	Memory grows with the tree; bound flags needed for cutoff values.
6	Manshaei et al. (2013)	Game Theory Meets Network Security and Privacy	Survey of attacker-defender game models (ACM CSUR)	Theoretical survey; few open, playable or explainable implementations.

Research Gap and Project Response

1. Literature findings
Minimax and Alpha-Beta solve two-player games; security games model attacker-defender conflict.

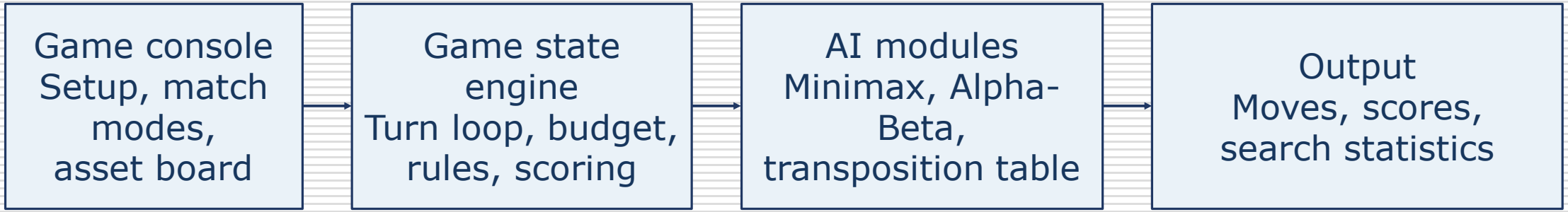
2. Limitations
Chess-style games ignore budgets; security models are rarely playable or explainable.

3. Refined research gap
One open prototype should turn adversarial search into a measurable security battle.

4. Progress toward the solution
All modules are implemented. Alpha-Beta matches Minimax with 95.8% fewer nodes on 8 assets.

System Architecture / Proposed System

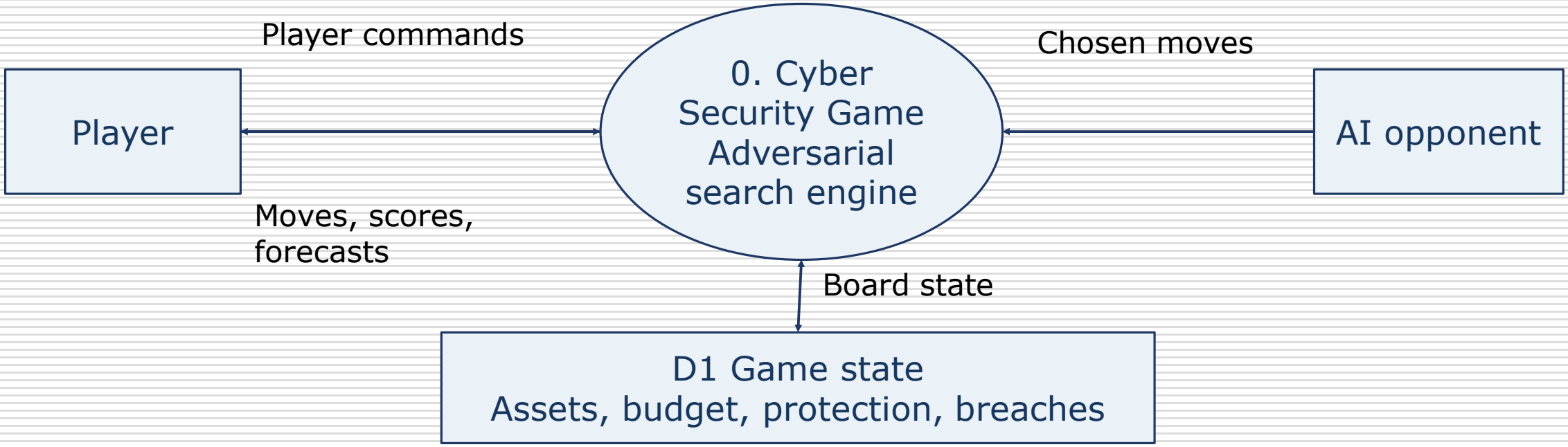
Proposed architecture



Rules live in a pure GameState class; search runs on cloned bitmask states, so hypothetical moves never corrupt the board.
The asset pool holds 22 realistic digital assets; every seed reshuffles a fresh 6 to 10 asset scenario.

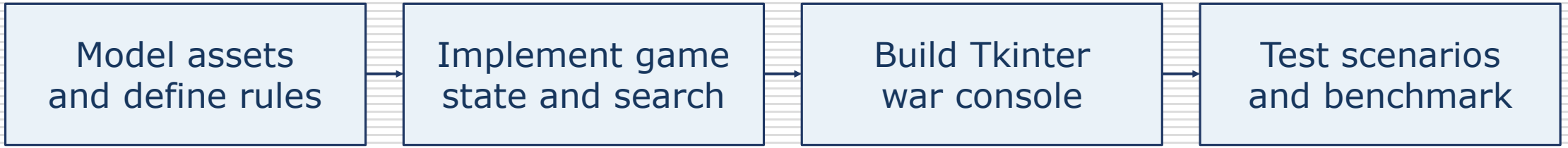
DFD Diagram

Level 0 context diagram

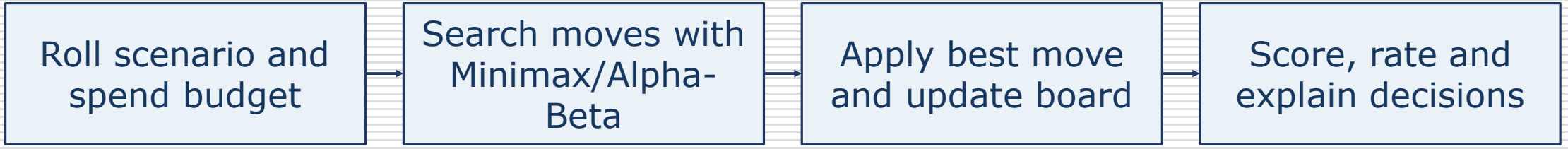


Methodology / Workflow Diagram

Development workflow



Runtime workflow



The worst-case damage readout re-runs Alpha-Beta after every move, showing the exact minimax forecast.

Tools and Technologies

- Python 3 implements the game rules, Minimax and Alpha-Beta search from scratch, with no AI libraries.
- Tkinter and ttk build the console: asset board, command menu, intel popups and game-over screen.
- Random and dataclasses shape seeded scenarios; difflib matches mistyped asset names.
- The engine counts nodes, cutoffs and cache hits; VS Code supports development.
- Standard laptop: 4 GB RAM minimum, no GPU. No database or machine learning is used.

List of Modules

- Asset and Scenario Model: Implemented and tested.
- Game State and Rules: Implemented and tested.
- Minimax Search: Implemented and tested.
- Alpha-Beta Search: Implemented and tested.
- Search Instrumentation: Implemented and tested.
- AI Turn Controller: Implemented and tested.
- Game Console and Popups: Implemented and tested.
- Game Over and Analysis: Implemented; screenshots pending.

Module Implementation – Part 1

- Module focus: Asset Model and Game Rules (Asset dataclass, GameState class).
- Input: asset pool, seed and asset count. Output: a 6 to 10 asset board with risk and a half-budget Defender.
- Risk = value x vulnerability; 22 realistic assets are sampled fresh for every seeded match.
- Pure rule engine: protect, scan, pass, attack, end turn; two no-progress turns end a stalemate.
- Scoring: Defender banks protected risk, Attacker banks breach damage; higher score wins the match.

Module Implementation – Part 2

- Module focus: Search Engine and AI Turn Controller (SearchEngine class).
- Minimax: the Defender minimises terminal damage and the Attacker maximises it over bitmask game states.
- Alpha-Beta: window pruning, high-risk-first ordering and EXACT, LOWER, UPPER bound caching.
- A damage-plus-exposed-risk heuristic bounds deep trees; search always runs on cloned states to protect the live board.
- Output feeds the console log, the worst-case readout and the AI Brain Battle chart.

Preliminary Results and Discussion

- Same start state, 8 assets, budget 4: Minimax 85,944 nodes; Alpha-Beta 3,644 (95.8% fewer) with 864 cutoffs.
- Both chose identical moves with the same worst-case damage (223.6).
- Scaling 6 to 10 assets lifted savings from 89.5% to 98.3% (0.26 s vs 5.7 s at 10 assets).
- Full AI vs AI match, seed 424242: Defender won 242.6 to 223.6 in 4 rounds on 5,495 nodes.
- Baseline: plain Minimax. The live worst-case forecast stays instant via Alpha-Beta after every move.

User Interface / Implementation Screenshots

- Menu: match mode (vs AI, local two-player, AI vs AI), role, difficulty, algorithm, seed and asset count.
- Console: asset board with exposed, scanned, protected or breached status, budget, scores, worst-case damage.
- Command menu: protect, scan, attack, pass and typed commands; asset intel popups show value, risk and status.
- Working application screenshots: [insert verified screenshots of the console and game-over screen].

Challenges Faced and Solutions

- Slow Expert search on early builds: high-risk-first ordering plus transposition caching cut nodes by 95.8%.
- Wrong winners after replay: scoring now counts only protected risk as Defender points, matching the scoreboard.
- Hypothetical moves corrupting the board: search runs on cloned states behind bitmask masks.
- Mistyped commands: fuzzy asset matching accepts partial names and typos.
- Remaining issue: benchmark 12+ asset boards and tune the Novice difficulty curve.

Work Plan Until Final Review

- Proposed Week 1: benchmark Minimax and Alpha-Beta on 12+ asset boards across many seeded matches.
- Proposed Week 2: hide true risks until scanned (partial observability) so information gathering has measurable value; compare difficulty win rates.
- Proposed Week 3: add chance events such as patch failures and zero-days to move toward expectiminimax; capture screenshots.
- Proposed Week 4: complete documentation, the final report and the demonstration rehearsal.
- Exact dates to be confirmed with the supervisor.

References

[1] Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
[2] Shannon, C. E. (1950). Programming a computer for playing chess. Philosophical Magazine, 41(314), 256–275.
[3] Knuth, D. E., & Moore, R. W. (1975). An analysis of alpha-beta pruning. Artificial Intelligence, 6(4), 293–326.
[4] Slate, D. J., & Atkin, L. R. (1977). CHESS 4.5 – The Northwestern University chess program. In Chess Skill in Man and Machine (pp. 82–118). Springer.
[5] Lye, K., & Wing, J. M. (2005). Game strategies in network security. International Journal of Information Security, 4(1–2), 71–86.
[6] Manshaei, M. H., Zhu, Q., Alpcan, T., Basar, T., & Hubaux, J.-P. (2013). Game theory meets network security and privacy. ACM Computing Surveys, 45(3), 1–39.
[7] Python Software Foundation. Python documentation. https://docs.python.org/
[8] Python Software Foundation. Tkinter documentation. https://docs.python.org/3/library/tkinter.html

Thank You

Cyber Security Game
Defender vs Attacker security strategy battle